

Smoothing to Fuctional Data in R

Statistical Computing / Functional Data Analysis

Task 1: multivariate depths

First let us analyse the multivariate depths in R package `DepthProc` function `depth`

Data Depth Functions

Let $X_1, \dots, X_n \in \mathbb{R}^d$ be a sample from a distribution F and let $x \in \mathbb{R}^d$ be a point whose depth relative to the sample is to be computed. Data depth provides a center-outward ordering of multivariate observations.

Projection Depth

Projection depth is based on the concept of projection pursuit and the Stahel–Donoho outlyingness. The outlyingness of a point x with respect to the distribution F is defined as

$$O(x; F) = \sup_{\|u\|=1} \frac{|u^T x - \text{Med}(u^T X)|}{\text{MAD}(u^T X)},$$

where u is a unit direction vector, Med denotes the median, and MAD denotes the median absolute deviation of the projected data.

The projection depth is then defined as

$$D_{\text{proj}}(x; F) = \frac{1}{1 + O(x; F)}.$$

Computational algorithm. In practice, the supremum is approximated using a finite number of random directions.

1. Generate N random unit vectors $u_1, \dots, u_N$.
2. For each direction u_i compute projections

$$z_j = u_i^T X_j, \quad j = 1, \dots, n.$$

3. Compute

$$m_i = \text{Med}(z_1, \dots, z_n), \quad s_i = \text{MAD}(z_1, \dots, z_n).$$

4. Compute standardized distance

$$o_i = \frac{|u_i^T x - m_i|}{s_i}.$$

5. Approximate outlyingness

$$O(x) \approx \max_{i=1, \dots, N} o_i.$$

6. The projection depth is

$$D_{\text{proj}}(x) = \frac{1}{1 + O(x)}.$$

Mahalanobis Depth

Mahalanobis depth is defined using the Mahalanobis distance from the mean vector. Let μ denote the mean of the distribution and Σ its covariance matrix.

The Mahalanobis distance is

$$MD(x) = \sqrt{(x - \mu)^T \Sigma^{-1} (x - \mu)}.$$

The Mahalanobis depth is defined as

$$D_{\text{Mah}}(x; F) = \frac{1}{1 + (x - \mu)^T \Sigma^{-1} (x - \mu)}.$$

Computational algorithm.

1. Compute the sample mean

$$\hat{\mu} = \frac{1}{n} \sum_{i=1}^n X_i.$$

2. Compute the sample covariance matrix

$$\hat{\Sigma} = \frac{1}{n-1} \sum_{i=1}^n (X_i - \hat{\mu})(X_i - \hat{\mu})^T.$$

3. Compute the inverse covariance matrix $\hat{\Sigma}^{-1}$.

4. For a query point x compute

$$d = (x - \hat{\mu})^T \hat{\Sigma}^{-1} (x - \hat{\mu}).$$

5. The Mahalanobis depth is

$$D_{\text{Mah}}(x) = \frac{1}{1 + d}.$$

Euclidean Depth

Euclidean depth is based on the Euclidean distance from a central point, typically the sample mean μ .

The Euclidean distance is

$$ED(x) = \|x - \mu\|.$$

The Euclidean depth is defined as

$$D_E(x) = \frac{1}{1 + \|x - \mu\|}.$$

Computational algorithm.

1. Compute the center $\hat{\mu}$ of the sample.
2. Compute the Euclidean distance

$$d = \sqrt{\sum_{i=1}^d (x_i - \mu_i)^2}.$$

3. The Euclidean depth is

$$D_E(x) = \frac{1}{1 + d}.$$

Tukey (Halfspace) Depth

The Tukey depth, also known as halfspace depth, measures the smallest probability mass in any halfspace containing the point x .

Formally, it is defined as

$$D_T(x; F) = \inf_{u \in S^{d-1}} P(u^T X \geq u^T x),$$

where S^{d-1} denotes the unit sphere in $\mathbb{R}^d$. For an empirical sample the depth becomes

$$D_T(x) = \min_u \frac{1}{n} \sum_{i=1}^n \mathbf{1}\{u^T X_i \geq u^T x\}.$$

Computational algorithm. Since exact computation is difficult in higher dimensions, an approximate algorithm based on random projections is used.

1. Generate N random directions $u_1, \dots, u_N$.
2. For each direction compute projections

$$z_i = u^T X_i, \quad z_x = u^T x.$$

3. Count observations on each side of the hyperplane

$$n_1 = \#\{i : z_i \leq z_x\}, \quad n_2 = \#\{i : z_i \geq z_x\}.$$

4. Compute directional depth

$$d_u = \min\left(\frac{n_1}{n}, \frac{n_2}{n}\right).$$

5. The Tukey depth is approximated by

$$D_T(x) \approx \min_u d_u.$$

The tasks

1. Depth for a Univariate Data Set

Consider the following one-dimensional data set representing exam scores:

```
scores <- c(45, 52, 60, 61, 65, 70, 72, 80, 85, 90)
```

- (a) Compute the depth of each observation with respect to the sample using the following methods:
 - Projection depth
 - Mahalanobis depth
 - Euclidean depth
 - Tukey depth
- (b) Store the results in a matrix.
- (c) Determine:
 - the maximal depth value,
 - the deepest observation,
 - the sample median.

- (d) Draw a plot showing:
- the observations on the horizontal axis,
 - their depth values on the vertical axis.
- (e) Compare the deepest observation with the sample median. Explain whether they coincide and why.

2. Depth Analysis for Simulated Data

Generate the following data sets:

```
set.seed(123)
sample1 <- rnorm(150, mean = 0, sd = 1)

set.seed(456)
sample2 <- runif(150, min = -2, max = 2)
```

- (a) For both data sets compute the Tukey depth of each observation with respect to the sample.
- (b) Determine for each sample:
- the maximal depth,
 - the deepest observation,
 - the sample median.
- (c) Draw the following graphs:
- a histogram of each sample,
 - a scatter plot of observations versus depth values.
- (d) On the histogram add vertical lines representing:
- the deepest observation,
 - the sample median.
- (e) Compare the results for the two distributions and answer:
- Is the deepest point always equal to the median?
 - Does the distribution shape influence depth values?

3. Multivariate Data Depth

Use the built-in R data set:

```
data(iris)
iris_data <- as.matrix(iris[,1:2])
```

This data set contains measurements of sepal length and sepal width for iris flowers.

- (a) Compute the following depth measures for the data set:
- Euclidean depth
 - Local depth
 - Modified Band Depth (MBD)
 - Fraiman–Muniz depth (FM)
- (b) Determine for each method:
- the maximal depth value,

- the deepest observation,
 - the coordinates of the deepest point.
- (c) Draw a scatter plot of the data and highlight:
- the deepest point according to Euclidean depth,
 - the deepest point according to Local depth,
 - the deepest point according to MBD,
 - the deepest point according to FM,
 - the coordinate-wise median point

$$(\text{median}(\text{iris_data}[, 1]), \text{median}(\text{iris_data}[, 2])).$$

- (d) Draw a local depth contour plot:

```
depthContour(iris_data,
depth_params = list(method = "Local",
depth_params1 = list(method = "LP")))
```

- (e) Compare the results of all methods and discuss:
- whether the deepest points coincide,
 - which methods give similar results,
 - how the deepest points relate to the center of the data cloud.

Task 2: Growth data

1. Compute mean, standard deviation and covariance for smoothed data.
2. Compute mean, standard deviation and covariance for the first derivative.
3. Compute mean, standard deviation and covariance for the second derivative.
4. Plot the results.

Task 3: PCA analysis

Interpretation of Functional Principal Components

Let $x_1(t), x_2(t), \dots, x_n(t)$ be a sample of functional observations defined on an interval $t \in [a, b]$. The mean function of the sample is defined as

$$\mu(t) = \frac{1}{n} \sum_{i=1}^n x_i(t).$$

Functional principal component analysis (FPCA) seeks orthonormal functions $\phi_1(t), \phi_2(t), \dots$ that maximize the variance of the projected functions. These functions satisfy the eigenvalue problem

$$\int_a^b C(s, t) \phi_k(s) ds = \lambda_k \phi_k(t),$$

where $C(s, t)$ is the covariance function

$$C(s, t) = \frac{1}{n} \sum_{i=1}^n (x_i(s) - \mu(s))(x_i(t) - \mu(t)).$$

The k -th principal component score for the i -th observation is

$$\xi_{ik} = \int_a^b (x_i(t) - \mu(t))\phi_k(t) dt.$$

Each function can then be approximated using the principal component expansion

$$x_i(t) \approx \mu(t) + \sum_{k=1}^K \xi_{ik}\phi_k(t).$$

Visualization of Functional Principal Components

To interpret the effect of the k -th principal component, one commonly plots:

- the mean function $\mu(t)$,
- the functions obtained by adding and subtracting a multiple of the k -th principal component function.

Specifically, the following curves are examined:

$$\mu(t) + c\sqrt{\lambda_k}\phi_k(t)$$

and

$$\mu(t) - c\sqrt{\lambda_k}\phi_k(t),$$

where

- $\phi_k(t)$ is the k -th principal component function,
- λ_k is the corresponding eigenvalue,
- c is a chosen constant (often $c = 1$ or $c = 2$).

These curves represent functions that deviate from the mean in the direction of the principal component. The magnitude $\sqrt{\lambda_k}$ reflects the variability explained by that component.

Thus, plotting

$$\mu(t), \quad \mu(t) + c\sqrt{\lambda_k}\phi_k(t), \quad \mu(t) - c\sqrt{\lambda_k}\phi_k(t)$$

illustrates how the k -th principal component modifies the shape of the mean function and helps interpret the main modes of variation in the functional data.

The tasks

1. For the data from Task 2, compute principal components (use 3 harmonics).
2. Plot the result.
3. What could be the interpretation?
4. Compute and plot rotated PC. Is the result better interpretable?
5. Do the same above tasks for the first derivative. Which is better interpretable?

Task 4: Boxplots and outliers

MUOD outliers in the fdaoutlier package

Source: <https://www.nature.com/articles/s41598-018-24874-2> and <https://arxiv.org/pdf/2105.05213>

Let

$$Y_1, Y_2, \dots, Y_n$$

be a sample of functional observations, where each curve is observed on the same grid

$$t_1, t_2, \dots, t_p \in I.$$

We write

$$Y_i = (Y_i(t_1), Y_i(t_2), \dots, Y_i(t_p)).$$

The MUOD (*Massive Unsupervised Outlier Detection*) method assigns to each curve three indices:

$$I_S(Y_i), \quad I_M(Y_i), \quad I_A(Y_i),$$

measuring, respectively, *shape*, *magnitude*, and *amplitude* outlyingness. Curves with unusually large values of these indices are flagged as outliers of the corresponding type.

Step 1: Pairwise linear comparison. For a fixed curve Y_i , MUOD compares it to every other curve Y_j through a simple linear regression model

$$Y_i(t_\ell) = \alpha_{ij} + \beta_{ij}Y_j(t_\ell) + \varepsilon_{ij}(t_\ell), \quad \ell = 1, \dots, p.$$

The estimated slope and intercept are

$$\hat{\beta}_{ij} = \frac{\text{cov}(Y_i, Y_j)}{s_{Y_j}^2}, \quad \hat{\alpha}_{ij} = \bar{x}_i - \hat{\beta}_{ij}\bar{x}_j,$$

where

$$\bar{x}_i = \frac{1}{p} \sum_{\ell=1}^p Y_i(t_\ell), \quad s_{Y_i}^2 = \frac{1}{p-1} \sum_{\ell=1}^p (Y_i(t_\ell) - \bar{x}_i)^2,$$

and

$$\text{cov}(Y_i, Y_j) = \frac{1}{p-1} \sum_{\ell=1}^p (Y_i(t_\ell) - \bar{x}_i)(Y_j(t_\ell) - \bar{x}_j).$$

The sample Pearson correlation between curves Y_i and Y_j is

$$\hat{\rho}(Y_i, Y_j) = \frac{\text{cov}(Y_i, Y_j)}{s_{Y_i}s_{Y_j}}, \quad s_{Y_i}, s_{Y_j} \neq 0.$$

These quantities are the building blocks of the MUOD indices.

Step 2: Shape index. The *shape index* of Y_i is defined by

$$I_S(Y_i) = \left| \frac{1}{n} \sum_{j=1}^n \hat{\rho}(Y_i, Y_j) - 1 \right|.$$

This index measures how different the *shape* of Y_i is from the rest of the sample. If Y_i has a shape similar to most other curves, then the average correlation is close to 1, and $I_S(Y_i)$ is small. Large values of $I_S(Y_i)$ indicate a *shape outlier*.

Step 3: Magnitude index. The *magnitude index* of Y_i is defined as

$$I_M(Y_i) = \left| \frac{1}{n} \sum_{j=1}^n \hat{\alpha}_{ij} \right|.$$

This index reflects vertical displacement relative to the bulk of the sample. If a curve tends to lie systematically above or below the other curves, then the regression intercepts $\hat{\alpha}_{ij}$ tend to be far from zero, producing a large value of $I_M(Y_i)$. Hence, large values of $I_M(Y_i)$ indicate a *magnitude outlier*.

Step 4: Amplitude index. The *amplitude index* of Y_i is defined as

$$I_A(Y_i) = \left| \frac{1}{n} \sum_{j=1}^n \hat{\beta}_{ij} - 1 \right|.$$

This index captures differences in scale or oscillation size. If the curve Y_i has a similar amplitude to the rest, then the regression slopes are close to 1 and $I_A(Y_i)$ is small. Large values of $I_A(Y_i)$ indicate an *amplitude outlier*.

Outlier classification. After computing

$$\{I_S(Y_i)\}_{i=1}^n, \quad \{I_M(Y_i)\}_{i=1}^n, \quad \{I_A(Y_i)\}_{i=1}^n,$$

MUOD determines a cutoff for each family of indices. In the ‘fdaoutlier’ package, the available cutoff rules are:

- `cut_method = "boxplot"`, based on a classical boxplot rule;
- `cut_method = "tangent"`, based on the tangent cutoff proposed with MUOD.

A curve Y_i is then classified as:

$$\begin{aligned} \text{shape outlier} &\iff I_S(Y_i) > c_S, \\ \text{magnitude outlier} &\iff I_M(Y_i) > c_M, \\ \text{amplitude outlier} &\iff I_A(Y_i) > c_A, \end{aligned}$$

where c_S, c_M, c_A are the cutoffs obtained from the chosen rule. The package returns the detected outliers and the three sets of indices.

Interpretation. Therefore, MUOD separates functional outlyingness into three components:

$$\text{outlyingness} = \text{shape} + \text{magnitude} + \text{amplitude}.$$

Informally:

- **Shape outlier:** a curve with an atypical pattern or form;
- **Magnitude outlier:** a curve shifted upward or downward;
- **Amplitude outlier:** a curve with unusually large or small scale.

This is the main idea behind the ‘muod()’ function in ‘fdaoutlier’.

The tasks

1. For the data from Task 2, draw boxplots. Are there any outliers?
2. Are there shape, magnitude and/or amplitude outliers?
3. For the data from Task 2, draw boxplots for derivatives. Are there any outliers?
4. Are there shape, magnitude and/or amplitude outliers in the derivatives?

R scripts

```
1 #####
2 # EDA for FDA: Smoothing Tasks
3 #####
4
5 library(fda)
6
7 #####
8 # TASK 1: Multivariate depth
9 #####
10 # Use codes from the lecture.
11
12 #####
13 # TASK 2: Growth data
14 #####
15 # Smoothing the growth data with a roughness penalty
16
17 # set up a basis for the growth data
18 # with knots at ages of height measurement
19
20 ageRng = c(1,18)
21 age = growth$age
22 agefine = seq(1,18,len=501)
23 heightmat = growth$hgtf
24
25 norder = 6
26 nbasis = length(age) + norder - 2
27 heightbasis = create.bspline.basis(ageRng, nbasis, norder, age)
28
29 heightLfd = 4
30 heightlambda = 10^(-3.75)
31 heightfdPar = fdPar(heightbasis, heightLfd, heightlambda)
32
33 # smooth the data
34
35 heightfdSmooth = smooth.basis(age, heightmat, heightfdPar)
36 heightfdpen = heightfdSmooth$fd
37
38 plot(heightfdpen)
39
40 # Further use classroom scripts.
41
42 #####
43 # TASK 3: PCA for growth data
44 #####
45 # Use classroom scripts.
46
47 #####
48 # TASK 4: Boxplot for growth data
49 #####
50 # Use classroom scripts.
```